

Las 3 Sintaxis

PMA: Pasaje de mensajes Asincronico (String hola) (no existe in/out)

PMS: Pasaje de mensajes Sincronico (String hola) (no existe in/out)

ADA: es Ada... (hola: String) (se tiene que especificar in/out)

-  **Asincrónico** → canal con **queue implícita**.
-  **Sincrónico** → canal **sin queue**, necesita que receptor y emisor estén **sincronizados**.

PMA

Se declaran los canales

Hay Queue Implícitas en los canales

Flexible en sintaxis

Send

Receive

Instrucción	Semántica
<code>chan nombre(tipoDato)</code> 	Canal único.
<code>chan arreglo[N](tipoDato)</code> 	Canales privados.
<code>send canal(dato)</code> 	Deja un dato en la cola implícita del canal. Es no bloqueante .
<code>send arreglo[i](dato)</code> 	Deja un dato en la cola implícita de un canal privado. Es no bloqueante . Cada pos, tiene un canal.. Cada canal tiene una queue implícita
<code>receive canal(variable)</code> 	Quita un elemento de la cola implícita del canal y lo pone en la variable. Es bloqueante .
<code>receive arreglo[i](variable)</code> 	Quita un elemento de la cola implícita del canal privado y lo pone en la variable. Es bloqueante .
<code>empty(canal)</code> 	Retorna V si la cola implícita del canal está vacía, F caso contrario.
<code>empty(arreglo[i])</code> 	Retorna V si la cola implícita del canal privado está vacía, F caso contrario.

PMA

- SEND
- NO BLOQUEANTE.

- RECEIVE
- BLOQUEANTE.

BARRERA

Seria como un vector de Queues

```
import java.util.*;

List<Queue<Integer>> colas = new ArrayList<>();

for (int i = 0; i < 10; i++) {
    colas.add(new LinkedList<>());
}

// Ejemplo de uso:
colas.get(0).add(3);
colas.get(0).add(6);
System.out.println(colas.get(0).poll()); // Imprime 3
```

```
pma

chan llegar(); // Canal común para avisar llegada
chan seguir[0..N-1](); // Canal individual para liberar a cada cliente

Process Cliente[id: 0..N-1] {
    send llegar(); // Aviso que llegué a la barrera
    receive seguir[id](); // Espero hasta que me den la orden de seguir
    // Luego sigue la ejecución normal...
}

Process Barrera {
    int contador = 0;

    while (contador < N) {
        receive llegar(); // Recibe llegada de un cliente
        contador++;
    }

    // Una vez llegaron todos, habilita a todos los clientes para continuar
    for i = 0 to N-1 {
        send seguir[i]();
    }
}
```

Cosas

PMA

-Potencialmente va a necesitar de un Coordinador -> para resolver su problema de isEmpty()

-- Al hacer un send es "no bloqueante" N procesos podrian hacer send al mismo tiempo, la estructura perse "canal" ya sabe manejar la proteccion paralela

--Aca usamos While (true)

-Cuando usar coordinador... por el problema de isEmpty ~~yyyyy cuando necesitas manejar "por orden de llegada"~~

```
1  expl
2  process coordinador{
3      while(true){
4          receive vendedorLibre(idVend);
5          if (pedido.empty()){
6              id=-1;
7          }else{
8              receive pedido(id,pedido);
9          }
10         send vendedor(id,pedido);
11     }
12 }
```

//EXPLICACION DEL USO DEL COORD: el proceso coordinador hace de pasamanos y regula el comportamiento de Los vendedores, esto pasa por que //Los vendedores no pueden evaluar de forma correcta que un canal sea empty, ya que puede pasar, que 2 procesos pregunten por //un canal que no sea vacio en un if(Lo cual es verdad) y al entrar haber un mensaje solo pudiendo suceder que uno de Los dos procesos lo //toma y el otro se queda esperando en el receive. Este 2do proceso no hace el delay al quedarse trabado, pero si ademas nunca Llegase //otro mensaje el proceso nunca finalizaria.

PMA

Cosas

1) En una oficina existen 100 empleados que envían documentos para imprimir en 5 impresoras compartidas. Los pedidos de impresión son procesados por orden de llegada y se asignan a la primera impresora que se encuentre libre:

- a) Implemente un programa que permita resolver el problema anterior usando PMA.
- b) Resuelva el mismo problema anterior pero ahora usando PMS.

A- PMA

//Esto tambien soluciona el problema, Seria la mejor solucion para el problema...

//si tuviera que la impresora 1 es la mas rapida "podria tomar siempre todos los pedidos" pero en la solucion se evita esto

chan empleadoPeticion(int, Documento); // Empleados envían doc + su id

chan recibirCopia[100](Documento); // Cada empleado tiene su canal para recibir la copia

```
Process Impresora(id: 1..5){
  Documento doc;
  int idE;
  while (true){
    receive empleadoPeticion(idE, doc);
    doc = imprimiendo(doc);
    send recibirCopia[idE](doc);
  }
}
```

```
Process Empleado(id: 1..100){
  Documento doc;
  while(true){
    send empleadoPeticion(id, doc);
    receive recibirCopia[id](doc);
  }
}
```

//=====con coordinador"innecesario"=====

1) La UNLP está organizando su maratón anual y P personas que deben pasar a retirar su remera y chip antes de la carrera. Para ello, los corredores deben dirigirse al edificio de Rectorado, donde habrá un organizador que los atenderá. Los corredores son atendidos de acuerdo con el orden de llegada, teniendo prioridad los corredores ancianos sobre los jóvenes y adultos. Adicionalmente, los corredores con discapacidad tienen prioridad sobre todos los anteriores. Para ser atendidos, los corredores entregan su DNI al organizador, quien como respuesta les entrega la remera oficial y el chip asociado. Implemente un programa que permita resolver el problema anterior usando **PMA**. **Notas:** para conocer su prioridad, cada corredor puede llamar a la función *obtenerPrioridad()*, la cual retorna 0 si la persona tiene alguna discapacidad, 1 si es anciana, o 2 en otro caso; la función *obtenerRyC(dni)* retorna la remera y el chip asociado para el DNI recibido como entrada; la función *obtenerDNI()* retorna el DNI para el corredor que la invoca.

```
chan atencionGeneral (int,int);
chan atencionConDiscapacidad (int,int);
chan atencionAnciano (int,int);
chan respuesta [P] (remera, chip);
chan señalizador ();
```

```
Process Corredor [i=0..P-1] {
  Remera r; Chip c;
  int prioridad = obtenerPrioridad();
  int dni = obtenerDNI();
  // según la prioridad, mando mensaje en canal correspondiente
  if (prioridad == 0)
    send atencionConDiscapacidad(i,dni);
  elif (prioridad == 1)
    send atencionAnciano(i,dni);
  else
    send atencionGeneral(i,dni);
  fi
  // envío señal para avisar que hay mensaje pendiente de atencion
  send señalizador();
  // espero respuesta en canal propio
  receive atencion[i] (r,c);
}
```

```
Process Organizador {
```

```
  Remera r; Chip c;
  int dni, id;
```

```
  while (true) {
    receive señalizador();
    if (not empty(atencionConDiscapacidad)) ->
      receive atencionConDiscapacidad(id,dni);
    // sólo atiendo si no hay mensajes de prioridad superior
    [] (not empty(atencionAnciano)) && (empty(atencionConDiscapacidad)) ->
      receive atencionAnciano(id,dni);
    // sólo atiendo si no hay mensajes de prioridad superior
    [] (not empty(atencionGeneral) && (empty(atencionAnciano)) &&
      (empty(atencionConDiscapacidad))) ->
      receive atencionGeneral(id,dni);
    fi
    (remera,chip) = obtenerRyC(dni);
    send respuesta[id] (remera,chip);
  }
}
```

No usar IF ELSE del lado del receptor.
Por que? nose no entiendo el if []
Solo tiene sentido por la parte
de que funciona como tomar una foto

PMS

$$B; C \rightarrow S$$

No Se declaran los canales

No hay Queue Implicitas en los canales

Flexible en sintaxis

No existe Send, Receive, Se usan etiquetas

Send = ! -> bloqueante

Receive = ? -> bloqueante

Tenemos nuevas estructuras de iteracion

DO OD

y

IF FI

Instrucción	Semántica
<code>procesoDestino!etiqueta(dato)</code>	Un proceso le envía un dato a otro. Es bloqueante .
<code>procesoDestino[i]!etiqueta(dato)</code>	Un proceso le envía un dato a otro específico dentro de un arreglo de procesos. Es bloqueante .
<code>procesoOrigen?etiqueta(variable)</code>	Un proceso se queda esperando a que el procesoOrigen le envíe un dato, el cual almacena en variable. Es bloqueante .
<code>procesoOrigen[i]?etiqueta(variable)</code>	Un proceso se queda esperando a que el procesoOrigen específico dentro del arreglo de procesos le envíe un dato, el cual almacena en variable. Es bloqueante .
<code>(Condición); Recepción -> Sentencias</code>	- La condición indica si el proceso está o no en condiciones de procesar el mensaje recibido en la Recepción. Puede omitirse en cuyo caso siempre es True. - La Recepción es justamente una sentencia de recepción (con el símbolo ?) que siempre debe estar. - Las Sentencias se ejecutarán luego de realizar la Recepción. - Cada guarda puede estar en 1 de 3 posibles estados: - Éxito: La condición es V o está omitida y la recepción puede realizarse en ese mismo instante sin demora alguna (el emisor ya estaba esperando hacer el envío). - Bloqueo: La condición es V o está omitida pero la recepción no puede realizarse de inmediato (el emisor aún no hizo el envío). - Fallo: La condición es F.
<pre>if [] [] fi do [] [] od</pre>	- En el If No Determinístico se evalúan todas las guardas al mismo tiempo y en base a eso: - Si una o más son exitosas se selecciona una de ellas en forma aleatoria, se realiza la recepción, y se ejecutan las sentencias. - Si ninguna guarda es exitosa pero hay una o más con estado de bloqueo entonces el proceso se demora en el If hasta que haya una exitosa, realiza la recepción, y ejecuta las sentencias. - El Do No Determinístico es igual que el If con la única diferencia de que en vez de realizar lo mencionado una sola vez, repite el mecanismo una y otra vez hasta que todas las guardas estén en estado de fallo.

- Guardas: cada alternativa es una guarda con la forma: **$B; C \rightarrow S$**
 - **B** : condición booleana que puede no estar (en ese caso se considera true), e indica si el proceso está o no en condiciones de procesar el mensaje recibido en **C** .
 - **C** : sentencia de comunicación (en la práctica sólo RECEPCIÓN) que seguro debe estar.
 - **S** : conjunto de sentencias que se ejecutarán en caso de ser elegida la guarda.

➤ Evaluación de una guarda:

- **Exito**: la condición booleana es Verdadera (o no la tiene) y la comunicación se puede realizar sin producir demora (el emisor está esperando hacer la comunicación).
- **Fallo**: la condición booleana es Falsa, sin importar lo que ocurra con la sentencia de comunicación.
- **Bloqueo**: la condición booleana es Verdadera (o no la tiene) pero la comunicación NO se puede realizar sin producir demora (el emisor aún no llegó a la sentencia de envío).

- En el **if “guardado”** se evalúan todas las guarda y en base a eso:
 - Si una o más son EXITOSAS se selecciona una de ellas en forma NO DETERMINISTICA, se ejecuta la sentencia de recepción (**C**) que forma parte de la guarda, y posteriormente el conjunto de sentencias asociadas a la guarda (**S**).
 - Si todas las guardas FALLAN no se selecciona ninguna y se sale del IF sin realizar ninguna acción.
 - Si no hay ninguna guarda EXITOSA pero hay una o más con estado BLOQUEO entonces el proceso se demora en el IF hasta que haya una guarda exitosa. En ese momento se ejecuta igual que el primer caso.

- El **do “guardado”** funciona de la misma manera que el IF, con la única diferencia que en lugar de hacerlo una vez repite el mecanismo hasta que todas las guardas FALLAN.

- Sentencias de comunicación: no existen las estructuras de canales, la comunicación se realiza nombrando al proceso con el cual se realiza la comunicación.

- Envío (!): la operación es bloqueante y sincrónica, se demora hasta que la recepción haya terminado.

destino!port (mensaje)

destino[i]!port (mensaje)

- Recepción (?): la operación es bloqueante y sincrónica.

origen?port (variable)

origen[i]?port (variable)

origen[]?port (variable)*

cada canal
No tiene una queue
implicita

4

PMS

¿Los canales en PMS (Pasaje de Mensajes Sincrónico) tienen una queue implícita como en el asincrónico?

➡ No, en el pasaje de mensajes sincrónico, no hay una queue implícita como en el asincrónico.

¿Por qué?

- En **pasaje sincrónico**, el envío de un mensaje **bloquea** al proceso emisor **hasta** que haya un receptor disponible. Es decir, **no se acumulan mensajes**: el canal **no almacena nada**, solo transmite en el momento en que ambos extremos están listos.
- En cambio, en el **pasaje asincrónico**, el canal sí puede tener una **cola implícita de mensajes**: el emisor manda y **no espera**, porque el mensaje queda almacenado en la queue hasta que el receptor lo procese.

PMS

Barrera

Barrera

```
process Cliente[id: 0..N-1] {  
  Barrera!!llegar()  
  Barrera?seguir()  
}  
  
process Barrera {  
  for i=0 to N-1  
    Cliente[*]?llegar()  
  for i=0 to N-1  
    Cliente[i]!seguir()  
}
```

✓ ¿Cómo funciona `do ... od`?

El bloque `do` contiene **guardas**, que son condiciones de habilitación para ejecutar una acción (como un `receive` o un `send`). Se comporta como un `while(true)` implícito, ejecutando alguna de las ramas habilitadas.

Ejemplo:

```
pma  
  
do  
  [] Persona[*]?quieroUsarTerminal(idP) ->  
    // lógica  
  [] Persona[*]?termineDeUsarTerminal() ->  
    // otra lógica  
od
```

 Copiar  Editar

Características del `do ... od`:

- Itera mientras alguna **guarda** esté habilitada.
- Si ninguna **guarda** puede ejecutarse, el proceso **bloquea** (espera).
- No necesitas escribir `while` ni `loop`: el `do` ya implica iteración.

PMS

Ejemplo
Exclusion mutua
Que NO
debe terminar

```
//En este caso si es necesario usar un coordinador.. En PMS nos encontramos con el problema de espera incensario de los mensajes...
//Empleado al hacer un send es bloqueante y no tiene una Queue implicita.. entonces retrasaria todo el pipeline
Process Empleado(id: 1..100){
    Documento doc;
    while(true){
        doc = generandoLoQueQuieroImprimir();
        Coordinador!estoyListo(id, doc);
        Impresora[*]?copiaLista(doc);
        mirandoLaDoc(doc);
    }
}

Process Impresora(id: 1..10){
    Documento doc; idE: integer;
    while(true){
        Coordinador.printerLista(id);
        Coordinador?esperandoDocumento(doc, idE);
        Documento impreso = imprimiendo(doc);
        Empleado[idE]?copiaLista(impreso);
    }
}

Process Coordinador { //como te dice que nunca debe terminar... el do od se va hacer infinitamente, por que la primera guarda[] su condicion siempre es true
    int idE, idI;
    Documento doc;
    Queue documentosEsperandoImpresion;
    do
        [] Empleado[*]?estoyListo(id, doc) ->
            documentosEsperandoImpresion.push(doc, idE);
        [] (!documentosEsperandoImpresion.isEmpty()); Impresora[*]?printerLista(idI) ->
            Impresora[idI]?esperandoDocumento(documentosEsperandoImpresion.pop());
    od
}
```

PMS

Ejemplo
Exclusion mutua
Que debe terminar

2) Resolver el siguiente problema con PMS. En la estación de trenes hay una terminal de SUBE que debe ser usada por P personas de acuerdo con el orden de llegada. Cuando la persona accede a la terminal, la usa y luego se retira para dejar al siguiente. Nota: cada Persona usa sólo una vez la terminal.

```
Process Persona(id: 1..P){  
  Coordinador!quieroUsarLaTerminal(id);  
  Coordinador?usarTerminal(estacion);  
  Coordinador!termineDeUsarTerminal(estacion);  
}
```

```
Process Coordinador{  
  boolean terminalLibre = true;  
  Queue fila;  
  integer cantPersonas = 0;  
  integer idP;  
  Terminal terminal; Integer cantTerminaron = 0;  
  do  
    [] (cantPersonas < P); Persona[*]?quieroUsarLaTerminal(idP) ->  
      if(terminalLibre == true){  
        terminalLibre = false;  
        Persona[idP]!usarTerminal(terminal);  
      }else{  
        fila.push(idP);  
      }  
      cantPersonas++;  
    [] (cantTerminaron < P); Persona[*]? termineDeUsarTerminal(estacion) ->  
      if(fila.isEmpty){  
        terminalLibre = true;  
      }else{  
        Persona[fila.pop()]!usarTerminal(estacion);  
      }  
      cantTerminaron++;  
  od  
}
```

"CADA PERSONA USA SOLO UNA VEZ LA TERMINAL"
LAS ITERACIONES SON FINITAS

PMS

Diferenciar ejercicios de exclusión mutua y los cliente-servidor

- En los ejercicios cliente-servidor suelen nombrar al servidor como un actor que "hace algo" mientras en los de exclusión mutua es un recurso que los clientes deben pedir y liberar. Por ej: en los C/S suele haber un empleado que atiende clientes, un profesor que corrige exámenes, etc. En cambio con exclusión mutua suele ser una cabina, un juego, una máquina que debe ser usada por los clientes pero que no realiza ninguna tarea por si misma.

```
//aca no tengo variables compartidas.. creo un proceso(o pongo en el Administrador los fors) que capture N mensajes de alumnos
Process Barrera(){
    for (int i = 0; i < N; i++){ // "duerme a todos" cualquier alumno que me haya mandado un mensaje lo duermo
        Alumno[*]?formandoBarrera();
    }

    for(int i = 0; i < N; i++){
        Alumno[i]!liberaronAtodos(); //voy liberando a los alumnos
        // Alumno[*]!liberaronAtodos(); el * es solo para recepcion
    }
}

Process Alumno(id: 0..N-1){
    //llego al examen
    //espero que todos lleguen
    //-----
    Barrera!formandoBarrera();
    Barrera?liberaronAtodos();
    //-----
    //haciendoExamen();
    Examen examen = haciendoExamen();
    Administrador!corregime(examen, id);
    Text examenCorregido;
    Profesor[*]?meCorrigio(examenCorregido); //de que profesor? de cualquiera
    //mirando Correcciones
}
```

ADA

No hay canales.. Aca se conocen como Entrys..

Aca Podemos tener conexiones bidireccionales

Cada Entry tiene una Queue implicita

No Flexible en sintaxis, usar la sintaxis de ada...

Loop = es un while true..

Ahora si quiero meterle una condicion hacer un while comun. NO LOOP(CONDCION), No existe

Instrucción	Semántica
<pre>Task tarea; Task tarea is Entry entry1; End tarea; Task tarea is Entry entry1(A, B: IN integer; C: OUT char; D: IN OUT boolean); End tarea;</pre>	Especificación de una tarea sin entrys, con entrys sin parámetros y con entrys con parámetros.
<pre>Task type tipoTarea is Declaración de los ENTRYs del tipo tarea End tipoTarea;</pre>	Especificación de un tipo tarea (para albergar N tareas iguales).
<pre>Task body tarea is Variables Begin Implementación de la tarea End tarea; Task body tipoTarea is Variables Begin Implementación del tipo tarea End tipoTarea;</pre>	Cuerpo de la tarea o tipo tarea.
<pre>arreglo: Array(1..10) of tipoTarea,</pre>	Arreglo de tareas.
<pre>Tarea1.nombreEntry; Tarea2.nombreEntry(5, x, y, z);</pre>	Especificación de un entry call sin parámetros y luego con parámetros.

Mas sintaxis ADA

```
//====  
  
vecBanco: array(1..20) of Banco;  
Task type Banco is  
  Entry miCotizacion(cotizacionCompra: out real, cotizacionVenta: out real);  
  Entry recibirId(id: in integer);  
end banco;  
  
task body Banco is  
  id: integer;  
  v,c : real;  
begin  
  Accept recibirId(pos: in integer)do  
    id = pos;  
  end recibirId;  
  loop  
    v = consultarApiCotizacionV(); --cuestionable... de esta forma no calcularia la cotizacion en el momento que t  
    c = consultarApiCotizacionV(); --consultar... en este caso creo que seria mejor calcular dentro del do... no m  
    Accept miCotizacion(cotizacionCompra: out real, cotizacionVenta: out real)do  
      -- cotizacionCompra = consultarApiCotizacionC(); --esto estaria mal, el llamado a la funcion puede tardar N  
      -- cotizacionVenta = consultarApiCotizacionV();  
      cotizacionCompra = c;  
      cotizacionVenta = v;  
    end do;  
  end loop;  
end Banco;  
  
begin  
  for i in (1..20)loop  
    vecBanco(i).recibirId(i);
```

Como yo declare los entrys..
yo solo puedo aceptarlos

-LOS ACCEPTS LOS HACE EL PROCESO QUE DECLARA LOS ENTRYS...

-EN EL ACCEPT SE NECESITA DECLARAR EL TIPO IN/OUT

-EL ACCEPT PUEDE HACER USO DEL DO "MANTIENE LA CONEXION HASTA Q SE TERMINE LO Q ESTE DENTRO"

```
Select  
  Tarea2.nombreEntry(5, x, y, z);  
  sentencias a realizar sólo si se completó el entry call (puede no haber ninguna)  
Else  
  sentencias a realizar solo si se canceló el entry call  
End select;
```

Especificación de un entry call condicional. No espera a que le acepten el pedido, si la otra tarea no está lista para realizar el accept a su pedido inmediatamente, entonces lo cancela (lo quita de la cola implícita) y realiza otra cosa.

```
Select  
  Tarea2.nombreEntry(5, x, y, z);  
  sentencias a realizar sólo si se completó el entry call (puede no haber ninguna)  
Or delay tiempo  
  sentencias a realizar solo si se canceló el entry call  
End select;
```

Especificación de un entry call temporal. Espera a lo sumo un tiempo a que la otra tarea realice el accept a su pedido, pasado el tiempo cancela el entry call (lo quita de la cola implícita) y realiza otra cosa.

```
Accept nombreEntry;  
  
Accept nombreEntry is  
  cuerpo del accept  
End nombreEntry;  
  
Accept nombreEntry(A, B: IN integer; C: OUT char; D: IN OUT boolean) is  
  cuerpo del accept  
End nombreEntry;
```

Especificación del accept sin cuerpo, con cuerpo, y con cuerpo y parámetros.

```
Select  
  Accept nombreEntry;  
  sentencias a realizar sólo si se completó el accept  
Or  
  When (condición) -> Accept nombreEntry2 is  
    cuerpo del accept  
    End nombreEntry2;  
  sentencias a realizar sólo si se completó el accept  
End select;
```

Select para los Accept. Permite tener varias alternativas de ACCEPT que pueden o no tener asociada una condición booleana (no se puede utilizar los parámetros del entry como parte de la condición) utilizando la clausula when.

Si un accept no tiene "when" entonces su condición es siempre true.

Elige entre 2 o más Accepts que estén en true y además tenga al menos 1 llamada pendientes de forma no determinística.

Se le puede agregar al final un else o un or delay a esta estructura.

Cosass

ADA

```
Begin
  pesoTotal = 0
  loop
    SELECT
      accept Salir(peso:IN int) do
        pesoTotal -= peso
      end Salir
    OR
      when (pesoTotal <= 4) =>
        accept IngresarAuto() do
          pesoTotal += 1
        end IngresarAuto
    OR
      when (pesoTotal <= 3) =>
        accept IngresarCamioneta() do
          pesoTotal += 2
        end IngresarCamioneta
    OR
      when (pesoTotal <= 2) =>
        accept IngresarCamion() do
          pesoTotal += 3
        end IngresarCamion
    END SELECT
  end loop
End Puente
```

```
TASK BODY Usuario is
  Documento : TEXT; OK: boolean;
Begin
  OK=false;
  Documento = new Documento();
  while (not OK) LOOP
    SELECT
      Servidor.peticionUsuario(Documento, OK);
    If (not OK) then
      Documento = ArreglarDocumento(Documento);
    end IF;
    OR DELAY 120.0;
    DELAY 60.0;
  end Select;
END LOOP;
END BODY Usuario;
```

```
2
3 TASK TYPE Usuario;
4 TASK BODY Usuario is
5   Documento : TEXT; OK: boolean;
6   Begin
7     OK=false;
8     Documento = new Documento();
9     while (not OK) LOOP
10      SELECT
11      Servidor.peticionUsuario(Documento, OK);
12      If (not OK) then
13      Documento = ArreglarDocumento(Documento);
14      end IF;
15      OR DELAY 120.0;
16      DELAY 60.0;
17      end Select;
18    END LOOP;
19  END BODY Usuario;
20
21  Begin
22    NULL;
23  end Ejercicio4.2;
```

~~EL SELECT SOLO VA DEL LADO DEL EMISOR DEL MENSAJE~~

Puede ir de ambos aldos

```
2
3 BEGIN
4   LOOP
5     SELECT
6       WHEN (AtenderPersona'COUNT = 0) → ACCEPT AtenderEnfermera (pedido: IN Pedido) DO
7         pedidoEnfermera = pedido;
8       END AtenderEnfermera;
9       ProcesarPedido (pedidoEnfermera);
10    OR
11      ACCEPT AtenderPersona DO
12      -- Atender Paciente;
13      END AtenderPersona;
14    ELSE
15      SELECT
16        Consultorio.RetirarNota(pedidoEnfermera);
17      IF (pedidoEnfermera != "") THEN
18        ProcesarPedido (pedidoEnfermera);
19      END IF;
20    ELSE
21      null;
22    END SELECT;
23  END SELECT;
24  END LOOP;
25  END Modulo;
```

Comportamiento del select

ADA

```
TASK Body Cliente IS
  soy_jubilado: boolean;
  nota: Reclamo;
  tram: Tramite;
  res: Resultado;
Begin
  soy_jubilado := SoyJubilado();
  tram := generarTramite();
  if (soy_jubilado) then
    SELECT
      empleadoBanco.atencionJub (tram,res);
    OR DELAY(15*60);
    nota := generarNota();
    responsableInformes.realizarReclamo(nota);
  END SELECT;
  else
    SELECT
      empleadoBanco.atencionNoJub (tram,res);
    OR DELAY(15*60);
    nota := generarNota();
    responsableInformes.realizarReclamo(nota);
  end SELECT;
  end if;
End Cliente;

BEGIN
  Null;
END Banco;
```

Ejemplo de `select` dentro de `else`:

```
ada

select
  Servidor.Atender;
  Put_Line("Llamada aceptada inmediatamente.");
else
  -- Si no se puede hacer el entry, hacé otro select con más opciones
  select
    delay 2.0;
    Put_Line("Esperé 2 segundos haciendo otra cosa.");
  or
    delay 5.0;
    Put_Line("O esperé 5 segundos.");
  end select;
end select;
```

```
task body Cliente is
begin
  loop
    select
      Servidor.Atender;
      Put_Line("Llamada aceptada inmediatamente.");
    else
      Put_Line("Servidor ocupado. Haciendo otra cosa...");
      delay 1.0;
    end select;
  end loop;
end cliente;
```

✓ Sintaxis básica

```
ada

select
  accept Operacion1;
  -- código
or
  accept Operacion2;
  -- otro código
or
  delay 10.0;
  -- código si ninguna de las anteriores llegó en 10 segundos
end select;
```

🔗 Cómo funciona

- El `select` evalúa todas las ramas `accept`.
- Si una está lista, se ejecuta inmediatamente.
- Si más de una está lista, elige una al azar (no determinismo).
- Si ninguna está lista, espera hasta que alguna lo esté.
- Si hay una rama `delay`, puede usarse como timeout.
- Si hay una rama `terminate`, sirve para finalizar una tarea cuando ya no se la necesita.

ADA

Comportamiento del select

Barrera

Sí, el `select` en Ada se ejecuta una sola vez cada vez que se lo alcanza en el flujo del programa.

¿Se repite solo?

✗ No, el `select` no itera automáticamente. Si querés que se repita, tenés que ponerlo explícitamente dentro de un bucle (`loop`).

✓ Ejemplo simple:

```
ada
-- Copiar  Editar

loop
  select
    Tarea.Entrada;
    Put_Line("Se hizo el accept.");
  else
    Put_Line("No se pudo hacer el accept.");
  end select;
end loop;
```

Acá el `select` se ejecuta en cada iteración del `loop`, y se comporta así:

- Si el entry está disponible al instante → entra por ese camino.
- Si no → entra por el `else`.

receptor

```
TASK TYPE Equipo IS
  ENTRY RecibirIdentificacion (nroId: IN int);
  ENTRY LlegadaBarrera;
  ENTRY SalidaBarrera;
  ENTRY SumarAlTotal (cantidadSumada: IN int);
END Equipo;
arrEquipos: array (1..5) of Equipo;
TASK BODY Equipo IS
  id, totalDeEquipo: int;
BEGIN
  totalDeEquipo = 0;
  ACCEPT RecibirIdentificacion (nroId: IN int) DO
    id = nroId;
  END RecibirIdentificacion;
  FOR i IN 1..4 LOOP
    ACCEPT LlegadaBarrera;
  END LOOP;
  FOR i IN 1..4 LOOP
    ACCEPT SalidaBarrera;
  END LOOP;
  FOR i IN 1..4 LOOP
    ACCEPT SumarAlTotal (cantidadSumada) DO
      totalDeEquipo = totalDeEquipo + cantidadSumada;
    END SumarAlTotal;
  END LOOP;
  Administrador.FinalizarEquipo (id, totalDeEquipo);
END Equipo;
```

Emisor

```
TASK TYPE Persona IS
  ENTRY RecibirIdentificacion(nroId: IN int);
END Persona;
arrPersonas: array (1..20) of Persona;
TASK BODY Persona IS
  totalEncontrado, equipo, equipoGanador: int;
BEGIN
  equipo = ...;
  totalEncontrado = 0;
  arrEquipos(equipo).LlegadaBarrera;
  arrEquipos(equipo).SalidaBarrera;
  FOR i in 1..15 LOOP
    totalEncontrado = totalEncontrado + Moneda();
  END LOOP;
  arrEquipos(equipo).SumarAlTotal (totalEncontrado);
  Administrador.ConsultarGanador (idEquipoGanador);
  IF (idEquipoGanador == idEquipo) THEN
    puts "A casa bots";
  END IF;
END Persona;
```

Resolver el siguiente problema con **ADA**. Simular el funcionamiento de un Entrenamiento de Básquet donde hay **20 jugadores** y **UN entrenador**. El entrenador debe distribuir a los jugadores en 2 canchas. Cuando un jugador llega el entrenador le indica la cancha a la cual debe ir para que se dirija a ella y espere hasta que lleguen los 10; en ese momento comienzan a jugar el partido que dura 40 minutos. Cuando ambos partidos han terminado, el entrenador les da a los 20 jugadores una charla de 10 minutos y luego todos se retiran. El entrenador asigna el número de cancha en forma cíclica 1, 2, 1, 2 y así sucesivamente.

PROCEDURE Entrenamiento is

```
Task type Jugador;
arrJugadores: array (1..20) of jugador;
```

```
Task type Cancha is
  entry llegue;
  entry iniciar;
  entry terminar;
```

```
End Cancha;
arrCanchas: array (1..2) of cancha;
```

```
Task Entrenador is
  entry Asignar (numC: out integer);
  entry TerminoPartido;
  entry Salir;
End Entrenador;
```

Task Body Entrenador is

```
Begin
  for C in 1..20 loop
    accept Asignar (numC: out integer) do
      numC = (C MOD 2) + 1;
    end Asignar;
  end loop;
  for i in 1..2 loop accept TerminoPartido; end loop;
  delay (10 minutos);
  for C in 1..20 loop accept Salir; end loop;
End Entrenador;
```

Task Body Jugador is

```
numC: integer;
Begin
  Entrenador.Asignar(numC);
  arrCanchas(numC).llegue;
  arrCanchas(numC).iniciar;
  arrCanchas(numC).terminar;
  Entrenador.Salir;
```

End Jugador;

Task Body Cancha is

```
Begin
  for i in 1..10 loop accept llegue; end loop;
  for i in 1..10 loop accept iniciar; end loop;
  delay (60 minutos);
  for i in 1..10 loop accept terminar; end loop;
  Entrenador.TerminoPartido;
End Cancha;
```

Begin

```
Null;
END Entrenamiento;
```

```
1 procedure Contador is
2   Task type Worker is
3     Entry Listo;
4     Entry Comenzar;
5     Entry AsignarID(id: in Integer);
6   end Worker;
7
8   Task type Coordinador is
9     Entry AvisarListo;
10    Entry Resultado(res: in Integer);
11  end Coordinador;
12
13  workers: array(1 .. 10) of Worker;
14  coordinador: Coordinador;
15
16  Task body Worker is
17    vector: array(1 .. 100000) of Float;
18    res: Integer := 0;
19  begin
20    accept AsignarID(id: in Integer) do
21      id_local := id;
22    end AsignarID;
23    coordinador.AvisarListo;
24    accept Comenzar;
25    res := SacarPromedio(vector);
26    coordinador.Resultado(res);
27  end Worker;
28
29  Task body Coordinador is
30    resultados: array(1 .. 10) of Float;
31    listos: Integer := 0;
32  begin
33    -- Esperar a que Los 10 workers estén Listos
34    for I in 1 .. 10 loop
35      accept AvisarListo;
36      listos := listos + 1;
37    end loop;
38    -- Todos Listos, ahora sí Los arrancamos
39    for I in 1 .. 10 loop
40      workers(I).Comenzar;
41    end loop;
42    -- Recolectar resultados
43    for I in 1 .. 10 loop
44      accept Resultado(res: in Integer) do
45        resultados(I) := res;
46      end Resultado;
47    end loop;
48  end Coordinador;
49  begin
50    for I in 1 .. 10 loop
51      workers(I).AsignarID(I);
52    end loop;
53  end Contador;
```

EJERCICIO 3

Procedure Banco3 is

```
Task empleado is
  Entry Prioritario (D: IN texto; R: OUT texto);
  Entry Regular (D: IN texto; R: OUT texto);
End empleado;

Task type cliente;
arrClientes: array (1..N) of Cliente;

Task Body cliente is
  Resultado: texto;
Begin
  if ("es cliente prioritario") then
    Empleado.Prioritario ("datos", Resultado);
  else
    Empleado.Regular ("datos", Resultado);
  end if;
End cliente;
```

Task Body empleado is

```
Begin
  loop
    SELECT
      accept Prioritario (D: IN texto; R: OUT texto) do
        R := resolverPedidoPrioritario(D);
      end Prioritario;
    OR
      when (Prioritario'count = 0) =>
        accept Regular (D: IN texto; R: OUT texto) do
          R := resolverPedidoRegular(D);
        end Regular;
    END SELECT;
  end loop;
end empleado;

Begin
  null;
End Banco3;
```

- Loop = es un while true..

Ahora si quiero meterle una condicion hacer un while comun. NO LOOP(CONDCION),

- No existe



```
begin
loop
continuar = false; intentos = 0;
loop (!continuar AND intentos < 3)
select
Medico.atenderPeticiónPersona; //requiero atención
continuar = false; //ya me atendió corto
or delay 60*5 //espero 5 minutos
intentos++;
if(intentos < 3)then
delay(60*10); //espero 10 minutos
end if;
end select; //solo se permite 1 select no se puede anidar selects
end loop;
loop end;
end Persona;
```

```
qr = obtenerQR()
while (intentos < 3) AND (atendido == false) loop
SELECT
-- Pedir atención
Emp.atencion(qr,paq);
atendido := true;

ELSE
-- dar vuelta a la plaza;
caminata();
-- cuenta el intento
intentos++;

End SELECT;
end loop
```